AURA-HOI: Exact High-Occupancy Itemset Mining on Dense Databases with a Length-Capped Residual Bound

Quan Van[1*]

[1*]Independent Researcher, Ho Chi Minh City, Vietnam.

Corresponding author(s). E-mail(s): vanhaminhquan2406@gmail.com;

Abstract

Exact miners of high-occupancy itemsets (HOIs) prune with an upper bound, because occupancy is not anti-monotone. The bound used by HEP and DFHOI depends only on the lengths of the supporting transactions. On dense databases it is close to the support, and equal to it when all transactions have the same length, so these miners nearly enumerate the frequent itemsets even when few or no HOIs exist. We present AURA-HOI, an exact depth-first miner over prefix equivalence classes of TID-lists, built around a length-capped residual bound that combines transaction lengths with the largest itemset the current branch can still generate. We prove that the bound is safe, never weaker than the transaction-length bound, and equal to it exactly when the branch can still produce itemsets as long as the supporting transactions, and that the miner is correct and complete. On seven benchmark datasets with the thresholds of the studies that introduced HEP and DFHOI, all programs return identical itemsets and agree with an exact rational-arithmetic oracle on 8,073 randomized cases. On the four dense datasets, AURA-HOI performs at least 84.9% fewer candidate intersections than DFHOI; where HOIs exist it is 4.86 to 64.2 times faster and completes settings that DFHOI and HEP do not, and where none exists it establishes this in at most 1.51 s. On the three sparse datasets, as the analysis predicts, it runs 0.92 to 1.07 times as fast as the faster baseline.

Keywords: High-occupancy itemset mining, Dense databases, Upper bound, Equivalence classes, Vertical mining, Search space pruning

1 Introduction

Frequent itemset mining finds the sets of items that co-occur in at least a given number of transactions [1, 2]. Its efficient algorithms, such as Apriori [2], FP-growth [3] and Eclat [4, 5], rely on the anti-monotonicity of support: every superset of an infrequent itemset is infrequent. Support alone, however, does not say how representative an itemset is of the transactions that contain it. An itemset of two items that occurs in many transactions of length fifty describes only a small part of each of them.

Tang et al. introduced *occupancy* to capture this aspect: the occupancy of an itemset relates its size to the sizes of the transactions that contain it, and patterns with high occupancy are better candidates for recommendation [6]. Deng formulated the mining of high-occupancy itemsets (HOIs) and proposed the HEP algorithm [7], and Nguyen et al. proposed DFHOI, a depth-first algorithm over equivalence classes with early pruning [8]. Occupancy has since been studied with transaction weights [9], in incremental settings [10], with adaptive thresholds [11], as a top-k problem [12], and combined with utility [13–15].

Occupancy is not anti-monotone: adding an item to an itemset increases its size but may shrink its set of supporting transactions, so the occupancy of the extension can be larger or smaller. Exact miners therefore prune with an *upper bound* on the occupancy of all extensions. The bound used by HEP and DFHOI, which we call the transaction-length bound, exploits the fact that an itemset cannot be larger than any transaction that contains it, and depends only on the lengths of the supporting transactions. It is effective when transactions are short compared with the itemsets one is looking for, which is the situation of sparse market-basket data. On dense databases it is weak. There, transactions are long and of similar length, and the bound is close to the support; when all transactions have the same length, as in many categorical benchmarks, it equals the support (Proposition 5). It then prunes nothing beyond the support constraint, and the miner enumerates every frequent itemset even when none of them has high occupancy. In our experiments, DFHOI performs about 1.6 million candidate intersections and runs for 298 s on the `accidents` dataset to find two HOIs, and HEP exceeds 20 GiB of memory; at the three larger thresholds, where no HOI exists, DFHOI needs up to 123 s to establish this.

A depth-first search over prefix equivalence classes provides a second, independent limit: inside a branch, an itemset can only be extended with the items that remain in its class, so its descendants have bounded size. This is the same principle as the remaining-utility bounds of high-utility itemset mining [16, 17]. On dense data the limit is informative: the items that reach the support threshold are few compared with the length of the transactions, so the classes of the search quickly become shorter than the transactions that support them. This paper combines the two limits in a single bound, builds an exact miner around it, and identifies precisely when the combination improves on the transaction-length bound. The main contributions are the following.

1. A *length-capped residual bound* on the occupancy of every itemset that can still be generated in a branch (Theorem 3). We prove that it is never weaker than the transaction-length bound or the purely residual bound (Proposition 4), and that it coincides with the transaction-length bound exactly when the branch can

still produce itemsets as long as every supporting transaction (Corollary 6). This characterizes where the bound helps: on dense databases, and not on sparse ones. It is computed in one linear pass over the TID-list of a candidate, and in constant time when transaction lengths are uniform.

2. AURA-HOI, a depth-first miner over prefix equivalence classes of TID-lists that applies the bound to nodes and to whole classes and allocates TID-lists from a stack-like arena. We prove that it returns exactly the set of HOIs (Theorem 7), and we show with a two-transaction counterexample that taking the residual length from the later siblings only, a tempting shortcut, loses results (Remark 1).
3. An experimental comparison with HEP and DFHOI on seven datasets, four dense and three sparse, with the thresholds of the studies that introduced the baselines: the five datasets of Deng [7] and the two further dense datasets of the DFHOI evaluation [8]. On the dense datasets AURA-HOI reduces the candidate intersections by at least 84.9%; where HOIs exist it reduces the runtime by factors of 4.86 to 64.2 and completes settings that the baselines do not, and where none exists it establishes this almost immediately; on the sparse datasets its runtime is close to that of the baselines, as Corollary 6 predicts. The itemsets returned by all programs are compared with each other and, on randomized databases, with an exact rational-arithmetic oracle, including cases in which an occupancy equals the threshold. These checks also reveal one HOI count in the literature that we could not reproduce (Section 6.3).

Section 2 reviews related work. Section 3 defines the problem. Section 4 develops the bounds and Section 5 the algorithm. Section 6 reports the experiments, and Section 7 concludes.

2 Related work

Frequent itemset mining.

Apriori generates candidates level by level and prunes them with the anti-monotonicity of support [1, 2]. FP-growth avoids candidate generation by recursively projecting a prefix tree [3]. Eclat represents each itemset by the sorted list of identifiers of its supporting transactions (its TID-list), groups itemsets that share a prefix into equivalence classes, and computes supports by intersecting TID-lists [4, 5]; diffsets reduce the size of these lists on dense data [18]. PrePost and PrePost+ use N-lists built from a prefix tree [19, 20]. Condensed representations, such as closed [21–23] and maximal [24] itemsets, reduce the number of reported patterns. Surveys of the field are available in [25–27].

Occupancy.

Tang et al. introduced occupancy as a measure of pattern quality [6]. Deng studied the mining of high-occupancy itemsets and proposed HEP, which stores occupancy lists and prunes with an upper bound derived from transaction lengths [7]. Nguyen et al. proposed DFHOI, which traverses equivalence classes depth-first and applies the bound early [8]. Datta et al. weighted occupancy by transaction occupancy [9]; Kim et al. mined high-occupancy patterns incrementally [10]; Tran et al. proposed a

hierarchical set-enumeration tree with an adaptive occupancy threshold that changes from level to level [11]; and Yıldırım addressed the top-k problem [12]. Occupancy has also been combined with utility [13–15]. NAM-HEP recomputes the occupancy threshold at every level of its search as the median occupancy of the candidates of that level [11], and the top-k formulation returns the k itemsets of largest occupancy [12]; both return result sets that differ from those of the fixed-threshold problem studied here, so we compare with HEP and DFHOI, which solve the same problem. When all transactions have the same length L, X is an HOI if and only if $\mathrm{supp}(X) \geq \xi L/|X|$, a support threshold that decreases with the length of the itemset; Seno and Karypis studied frequent itemsets under such length-decreasing support constraints and pruned with the smallest length that an extension must reach [28]. Our bound exploits the same relation between size and support through the residual length of a branch.

Bounds for measures that are not anti-monotone.

High-utility itemset mining faces the same difficulty, because utility is not anti-monotone either [29, 30]. HUI-Miner bounds the utility of the extensions of an itemset by its utility plus the remaining utility of the items that can still be appended [16]; FHM adds co-occurrence pruning [31]; and EFIM uses sub-tree and local utility bounds [17]. The residual part of our bound follows the same principle: in a depth-first enumeration, the items that can still be appended are known, which limits what the descendants can achieve. HUOPM transfers remaining utility to utility occupancy, per transaction [13]; for occupancy, the corresponding bound lets each supporting transaction assume all of its remaining items, which requires the number of remaining items of every transaction. Our bound uses instead a single residual length for the whole branch, which a TID-list miner knows without inspecting the transactions, and combines it with transaction lengths. Neither bound is always smaller than the other. With $|X| = 1$, a single supporting transaction of length 10 that contains 1 of $r = 5$ remaining items gives $2/10$ for the per-transaction bound and $U_6 = 6/10$ for ours; with $|X| = 1$, $r = 9$ and two supporting transactions of lengths 2 and 10 that contain all their remaining items, the per-transaction bound is $1 + 1 = 2$ and $U_{10} = 2 \cdot (1/2 + 1/10) = 6/5$.

3 Problem definition

Let I be a finite set of items and $D = \langle t_1, \ldots, t_n \rangle$ a database of n non-empty transactions $t_q \subseteq I$. For an itemset $X \subseteq I$, the set of supporting transactions and the support are

$$T(X) = \{q : X \subseteq t_q\}, \qquad \mathrm{supp}(X) = |T(X)|. \tag{1}$$

We write $|t_q|$ for the length of transaction t_q.

Definition 1 (Occupancy) The (summed) occupancy of an itemset X is

$$\mathrm{occ}(X) = \sum_{q \in T(X)} \frac{|X|}{|t_q|} = |X| \cdot S(X), \qquad S(X) = \sum_{q \in T(X)} \frac{1}{|t_q|}. \tag{2}$$

Table 1 Running example database with $n = 5$ transactions.

q	t_q	$\lvert t_q \rvert$	$1/\lvert t_q \rvert$
1	$\{A, B, C, D\}$	4	1/4
2	$\{A, B, C\}$	3	1/3
3	$\{A, B, D\}$	3	1/3
4	$\{A, C, D, E\}$	4	1/4
5	$\{B, C, D, E\}$	4	1/4

Each term $|X|/|t_q|$ is the fraction of transaction t_q that X occupies. We call $S(X)$ the reciprocal sum of X.

Definition 2 (High-occupancy itemset) Given a relative threshold $\alpha \in (0, 1)$, let $\xi = \alpha n$. An itemset $X \neq \emptyset$ is a high-occupancy itemset (HOI) if $\mathrm{occ}(X) \geq \xi$. The problem is to find the set $\mathcal{H}$ of all HOIs.

This is the fixed-threshold problem addressed by HEP [7] and DFHOI [8]. Our implementations of both baselines solve exactly this problem, which we verify against an exact oracle in Section 6.3.

Proposition 1 *For every itemset X, $\mathrm{occ}(X) \leq \mathrm{supp}(X)$. Hence every HOI has support at least $\sigma = \lceil \xi \rceil$.*

Proof If $q \in T(X)$ then $X \subseteq t_q$, so $|X| \leq |t_q|$ and each term of (2) is at most 1. There are $\mathrm{supp}(X)$ terms. Since $\mathrm{supp}(X)$ is an integer, $\mathrm{supp}(X) \geq \xi$ implies $\mathrm{supp}(X) \geq \lceil \xi \rceil$. □

Support is anti-monotone, so Proposition 1 gives a first pruning rule: no superset of an itemset with support below σ is an HOI. Occupancy itself is not anti-monotone, as the following example shows.

Example 1 Consider the database of Table 1 with $\alpha = 0.35$, so that $\xi = 7/4$ and $\sigma = 2$. For $X = \{A\}$, $T(X) = \{1, 2, 3, 4\}$ and $\mathrm{occ}(X) = 1 \cdot (1/4 + 1/3 + 1/3 + 1/4) = 7/6 < 7/4$, so $\{A\}$ is not an HOI. Its superset $\{A, B\}$ has $T = \{1, 2, 3\}$ and $\mathrm{occ}(\{A, B\}) = 2 \cdot (1/4 + 1/3 + 1/3) = 11/6 \geq 7/4$, so it is an HOI. An itemset that fails the threshold can therefore have supersets that pass it.

4 Upper bounds on occupancy

All bounds in this section bound the occupancy of every itemset Y that contains a given itemset X and has at most K items. In the search of Section 5, X is the current node and K is the size of the largest itemset that can still be generated from it.

Lemma 2 *If $X \subseteq Y$, then (i) $T(Y) \subseteq T(X)$, and (ii) $|Y| \le |t_q|$ for every $q \in T(Y)$.*

Proof (i) Every transaction that contains Y contains X. (ii) If $q \in T(Y)$ then $Y \subseteq t_q$. □

For an itemset X, let $\Lambda(X) = \{|t_q| : q \in T(X)\}$ be the set of lengths of its supporting transactions, and for an integer $\lambda \ge 1$ let

$$W_\lambda(X) = \sum_{q \in T(X),\ |t_q| \ge \lambda} \frac{1}{|t_q|} \tag{3}$$

be the reciprocal sum restricted to transactions of length at least λ. Note that $W_1(X) = S(X)$.

Definition 3 (Length-capped residual bound) For an itemset X with $T(X) \neq \emptyset$ and an integer $K \ge 1$,

$$U_K(X) = \max_{\lambda \in \Lambda(X)} \min(\lambda, K) \cdot W_\lambda(X), \tag{4}$$

and $U_K(X) = 0$ if $T(X) = \emptyset$.

Theorem 3 (Safety) *Let $X \subseteq Y$ and $|Y| \le K$. Then $\mathrm{occ}(Y) \le U_K(X)$.*

Proof If $T(Y) = \emptyset$ then $\mathrm{occ}(Y) = 0 \le U_K(X)$. Otherwise let $\ell = |Y|$. By Lemma 2, every $q \in T(Y)$ belongs to $T(X)$ and satisfies $|t_q| \ge \ell$. Hence

$$\mathrm{occ}(Y) = \ell \sum_{q \in T(Y)} \frac{1}{|t_q|} \le \ell \sum_{q \in T(X),\ |t_q| \ge \ell} \frac{1}{|t_q|} = \ell \cdot W_\ell(X). \tag{5}$$

The set $\{\lambda \in \Lambda(X) : \lambda \ge \ell\}$ is not empty, because it contains the length of any transaction in $T(Y)$. Let λ^* be its minimum. No transaction in $T(X)$ has a length in $[\ell, \lambda^*)$, so $W_\ell(X) = W_{\lambda^*}(X)$. Moreover $\ell \le \lambda^*$ and $\ell \le K$, so $\ell \le \min(\lambda^*, K)$. Together with (5),

$$\mathrm{occ}(Y) \le \min(\lambda^*, K) \cdot W_{\lambda^*}(X) \le U_K(X).$$

□

Two special cases are known. Without a size limit ($K \ge \max \Lambda(X)$), the bound becomes

$$\mathrm{UBO}(X) = \max_{\lambda \in \Lambda(X)} \lambda \cdot W_\lambda(X), \tag{6}$$

which depends only on transaction lengths. This is the upper bound on occupancy used by HEP and DFHOI [7, 8]; we refer to it as the transaction-length bound. Ignoring transaction lengths gives the purely residual bound $K \cdot S(X)$, which follows from $|Y| \le K$ and $T(Y) \subseteq T(X)$ alone.

Proposition 4 (Dominance) *For every itemset X and integer $K \ge 1$, $U_K(X) \le \min\{\mathrm{UBO}(X),\ K \cdot S(X)\}$, and $U_K(X)$ is non-decreasing in K.*

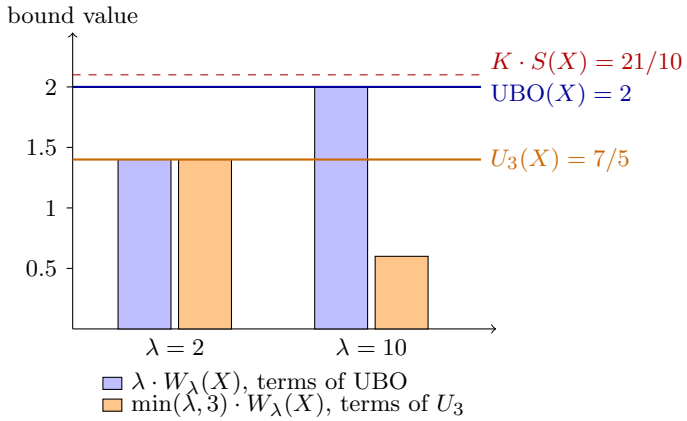


Fig. 1 The bounds for an itemset X supported by one transaction of length 2 and two of length 10, with $K = 3$: $W_2(X) = 7/10$ and $W_{10}(X) = 2/10$. The transaction-length bound UBO is attained at $\lambda = 10$, a length that no itemset of the branch can reach; capping λ at K reduces that term from 2 to 3/5, and the maximum moves to $\lambda = 2$. The purely residual bound $K \cdot S(X)$ ignores transaction lengths. $U_3(X) = 7/5$ is below both.

Proof For every $\lambda \in \Lambda(X)$, $\min(\lambda, K) \cdot W_\lambda(X) \le \lambda \cdot W_\lambda(X) \le \mathrm{UBO}(X)$, and $\min(\lambda, K) \cdot W_\lambda(X) \le K \cdot W_\lambda(X) \le K \cdot S(X)$ because $W_\lambda(X) \le W_1(X) = S(X)$. Taking the maximum over λ gives the first claim. Each term $\min(\lambda, K) \cdot W_\lambda(X)$ is non-decreasing in K, and so is their maximum. □

The bound can be strictly smaller than both special cases. If $T(X)$ consists of one transaction of length 2 and two of length 10, and $K = 3$, then $S(X) = 7/10$, $\mathrm{UBO}(X) = \max(2 \cdot 7/10,\ 10 \cdot 2/10) = 2$, $K \cdot S(X) = 21/10$, and $U_3(X) = \max(2 \cdot 7/10,\ 3 \cdot 2/10) = 7/5$ (Figure 1).

Example 2 In Example 1, consider $X = \{A, C, D\}$ with $T(X) = \{1, 4\}$, so $\Lambda(X) = \{4\}$, $W_4(X) = 1/2$ and $\mathrm{supp}(X) = 2 \ge \sigma$. The transaction-length bound is $\mathrm{UBO}(X) = 4 \cdot 1/2 = 2 \ge \xi = 7/4$, so it cannot prune X. In the search of Section 5 (Example 3), $\{A, C, D\}$ is generated as a child of $\{A, C\}$, and the largest itemset that can still contain it in that subtree has $K = 3$ items. Then $U_3(X) = \min(4, 3) \cdot 1/2 = 3/2 < 7/4$, and X is pruned safely: indeed $\mathrm{occ}(\{A, C, D\}) = 3/2$, and every proper superset of $\{A, C, D\}$ has support $1 < \sigma$.

The next result explains why the transaction-length bound fails on databases whose transactions have equal length.

Proposition 5 (Uniform lengths) *If every transaction has length L, then for every itemset X with $T(X) \neq \emptyset$*

$$\mathrm{UBO}(X) = \mathrm{supp}(X) \qquad \textit{and} \qquad U_K(X) = \frac{\min(L, K)}{L} \cdot \mathrm{supp}(X).$$

Proof Here $\Lambda(X) = \{L\}$ and $W_L(X) = \mathrm{supp}(X)/L$, so the maximum in (4) has a single term, $\min(L, K) \cdot \mathrm{supp}(X)/L$, which equals $\mathrm{supp}(X)$ when $K \geq L$. □

Since every itemset reached by a miner already satisfies $\mathrm{supp}(X) \geq \sigma \geq \xi$, the transaction-length bound never prunes on such databases, whereas $U_K(X) < \xi$ holds as soon as $K < L\xi/\mathrm{supp}(X)$.

The next corollary states, for any database, when the size limit adds nothing to the transaction-length bound.

Corollary 6 (Scope of the bound) *For every itemset X with $T(X) \neq \emptyset$, $U_K(X) = \mathrm{UBO}(X)$ if and only if $K \geq \lambda$ for some $\lambda \in \Lambda(X)$ that attains the maximum in (6). In particular, $U_K(X) = \mathrm{UBO}(X)$ whenever $K \geq \max \Lambda(X)$.*

Proof If $K \geq \lambda$ for a maximizer λ, then $U_K(X) \geq \min(\lambda, K) \cdot W_\lambda(X) = \lambda \cdot W_\lambda(X) = \mathrm{UBO}(X)$, and $U_K(X) \leq \mathrm{UBO}(X)$ by Proposition 4. Conversely, if $K < \lambda$ for every maximizer λ, then each term of (4) is smaller than $\mathrm{UBO}(X)$: for a maximizer λ, $\min(\lambda, K) \cdot W_\lambda(X) = K \cdot W_\lambda(X) < \lambda \cdot W_\lambda(X)$ because $W_\lambda(X) > 0$; for any other λ, $\min(\lambda, K) \cdot W_\lambda(X) \leq \lambda \cdot W_\lambda(X) < \mathrm{UBO}(X)$. Hence $U_K(X) < \mathrm{UBO}(X)$. □

The size limit therefore improves on the transaction-length bound only in branches that cannot produce itemsets as long as the supporting transactions that determine UBO. In the search of Section 5, the root class contains every item with support at least σ, and each level of the search shortens the classes. On a dense database, the frequent items are few compared with the length of the transactions, and this condition is met after few levels; on a sparse database, the classes remain much longer than the transactions, and the two bounds coincide in most of the search. Section 6 confirms both cases.

Computation.

Before mining, we re-index the transactions in non-decreasing order of length, as DFHOI does [8], so that sorted TID-lists are also sorted by transaction length. For a TID-list sorted in this way, a single backward scan accumulates $W_\lambda(X)$ for all $\lambda \in \Lambda(X)$ in decreasing order of λ, and the maximum in (4) is updated at each change of length. The bound therefore costs $O(|T(X)|)$; AURA-HOI computes it, together with $S(X)$, in one backward pass over the TID-list produced by the intersection. With uniform lengths, Proposition 5 gives it in $O(1)$. The purely residual bound $K \cdot S(X)$ costs $O(1)$ once $S(X)$ is known.

5 The AURA-HOI algorithm

5.1 Search space

The items with support at least σ are ordered by increasing support, ties being broken by item identifier, as in HEP [7, Section 3.2.3]; the examples order items of equal support alphabetically. As in Eclat [5], the search is organized in prefix equivalence

classes. A class is an ordered list $E = \langle N_0, \ldots, N_{k-1} \rangle$ of nodes that share a prefix P; node N_i stores the itemset $X_i = P \cup \{x_i\}$, its TID-list $T(X_i)$ in increasing order and its reciprocal sum $S(X_i)$. The children of N_i form the class of prefix X_i, whose candidates are $X_i \cup \{x_j\}$ for $j > i$, with $T(X_i \cup \{x_j\}) = T(X_i) \cap T(X_j)$. Without pruning, this enumeration generates every non-empty itemset exactly once [5].

For node N_i we call $r_i = k - 1 - i$ its residual length: the number of items that can still be appended to X_i in its own subtree. Every itemset generated in the subtree of N_i has at most $|X_i| + r_i$ items.

5.2 Pruning rules

Let $E = \langle N_0, \ldots, N_{k-1} \rangle$ be a class, $X = X_i$ the node being expanded, and $Y = X \cup \{x_j\}$ $(j > i)$ a candidate child. AURA-HOI applies three rules.

R1 (support) Discard Y if $\mathrm{supp}(Y) < \sigma$.

R2 (node bound) Discard Y if $U_K(Y) < \xi$ with $K = |Y| + r_i - 1$.

R3 (class bound) Do not expand X if $(|X| + r_i) \cdot S(X) < \xi$. After the children of X have been generated, do not recurse into them if $(|X| + c) \cdot S(X) < \xi$, where c is the number of children that survived R1 and R2.

Rules R1 and R3 cost $O(1)$. When all transactions have the same length L, AURA-HOI evaluates R2 and R3 with the closed form of Proposition 5: the value $(|X| + K') \cdot S(X)$ in R3 is replaced by $U_{|X|+K'}(X) = \min(L, |X| + K') \cdot \mathrm{supp}(X)/L$ for $K' \in \{r_i, c\}$, which is never larger by Proposition 4.

Rule R2 deserves care. A discarded child is removed from the class of prefix X altogether. It is therefore lost not only as the root of its own subtree but also as an item that the *earlier* children of X could append. The itemsets that contain Y and could still be generated in the subtree of X have the form $X \cup \{x_j\} \cup Z$ with $Z \subseteq \{x_l : l > i,\ l \neq j\}$, and there are $r_i - 1$ such items x_l. This is why R2 uses $K = |Y| + r_i - 1$, which counts all other children of X, and not only the items after x_j.

Remark 1 (A tempting but unsound shortcut) Using $K = |Y| + (k - 1 - j)$ in R2, that is, counting only the items that follow x_j, loses HOIs. Let $D = \langle \{A, B, C\}, \{A\} \rangle$ and $\alpha = 2/5$, so that $\xi = 4/5$ and $\sigma = 1$. The supports of A, B, C are 2, 1, 1, so the item order is $B \prec C \prec A$. The itemset $\{A, B, C\}$ is an HOI, since $\mathrm{occ}(\{A, B, C\}) = 3 \cdot 1/3 = 1$. In the class of prefix $\{B\}$, the candidate $Y = \{B, A\}$ comes last, so the shortcut uses $K = 2$ and obtains $U_2(Y) = \min(3, 2) \cdot 1/3 = 2/3 < \xi$. It discards $\{B, A\}$, and $\{B, C, A\}$, which would be generated from $\{B, C\}$ and $\{B, A\}$, is never produced. Rule R2 uses $K = 3$ instead, obtains $U_3(Y) = 1 \geq \xi$, keeps $\{B, A\}$ and finds $\{A, B, C\}$.

Example 3 We run Algorithm 1 on Table 1 with $\alpha = 0.35$ ($\xi = 7/4$, $\sigma = 2$). The supports of E, A, B, C, D are $2, 4, 4, 4, 4$, so the item order is $E \prec A \prec B \prec C \prec D$. Every singleton passes R1 and the transaction-length bound, and none is an HOI (the largest occupancy of a singleton is $7/6$), so $E_0 = \langle E, A, B, C, D \rangle$. Figure 2 shows the search.

Algorithm 1 AURA-HOI

Require: database D, threshold $\alpha \in (0, 1)$
Ensure: the set $\mathcal{H}$ of all HOIs

$\xi \leftarrow \alpha n$; $\sigma \leftarrow \lceil \xi \rceil$; re-index transactions by non-decreasing length
$E_0 \leftarrow \langle$ nodes $\{x\}$ with $\mathrm{supp}(\{x\}) \geq \sigma$ and $\mathrm{UBO}(\{x\}) \geq \xi \rangle$ ▷ R1 and Theorem 3
emit every $\{x\} \in E_0$ with $\mathrm{occ}(\{x\}) \geq \xi$; MINE($E_0$)
procedure MINE($E = \langle N_0, \ldots, N_{k-1} \rangle$)
    for $i = 0, \ldots, k-2$ **do**
        $X \leftarrow X_i$; $r \leftarrow k - 1 - i$
        if $(|X| + r) \cdot S(X) < \xi$ **then continue** ▷ R3
        $m \leftarrow$ ARENAMARK; $C \leftarrow \langle\,\rangle$
        for $j = i + 1, \ldots, k-1$ **do**
            $T \leftarrow T(X) \cap T(X_j)$ ▷ TID-list stored in the arena
            if $|T| < \sigma$ **then continue** ▷ R1
            compute $S(Y)$ and $U_{|Y|+r-1}(Y)$ for $Y = X \cup \{x_j\}$ in one pass over T
            if $U_{|Y|+r-1}(Y) < \xi$ **then continue** ▷ R2
            if $|Y| \cdot S(Y) \geq \xi$ **then** emit Y
            append the node of Y to C
        if $|C| \geq 2$ **and** $(|X| + |C|) \cdot S(X) \geq \xi$ **then** MINE(C) ▷ R3
        ARENAREWIND(m)

- $\{E\}$, $r = 4$: R3 allows the expansion, since $(1 + 4) \cdot 1/2 = 5/2 \geq \xi$. The candidates $\{E, A\}$ and $\{E, B\}$ have support 1 and fail R1; $\{E, C\}$ and $\{E, D\}$, with $T = \{4, 5\}$, pass R2 ($K = 5$, $U_5 = 2$) but have occupancy 1. With $c = 2$ surviving children, $(1 + 2) \cdot 1/2 = 3/2 < \xi$, so R3 does not recurse into them, and $\{E, C, D\}$ is never formed.
- $\{A\}$, $r = 3$: the children $\{A, B\}$ ($T = \{1, 2, 3\}$, occ $= 11/6$), $\{A, C\}$ and $\{A, D\}$ (occ $= 5/3$) pass R1 and R2 with $K = 4$; $\{A, B\}$ is emitted. In the class of $\{A, B\}$, the children $\{A, B, C\}$ ($T = \{1, 2\}$) and $\{A, B, D\}$ ($T = \{1, 3\}$) have occupancy $3 \cdot (1/4 + 1/3) = 7/4 = \xi$ and are emitted; their intersection $\{A, B, C, D\}$ has support 1. In the class of $\{A, C\}$, the candidate $\{A, C, D\}$ is discarded by R2 (Example 2).
- $\{B\}$, $r = 2$: $\{B, C\}$ and $\{B, D\}$ (occ $= 5/3$) are kept; their child $\{B, C, D\}$ ($T = \{1, 5\}$) has UBO $= 2 \geq \xi$ but $U_3 = 3/2 < \xi$ and is discarded by R2.
- $\{C\}$, $r = 1$: $\{C, D\}$ ($T = \{1, 4, 5\}$) has UBO $= 3$ but $U_2 = 2 \cdot 3/4 = 3/2 < \xi$, and is discarded by R2.

The output is $\{A, B\}$, $\{A, B, C\}$, $\{A, B, D\}$, which is the set of HOIs. Two of them have occupancy exactly ξ, so an inexact comparison could lose them. With the transaction-length bound alone, the nodes $\{C, D\}$, $\{A, C, D\}$ and $\{B, C, D\}$ would be kept, and $\{E\}$ would be expanded into the class of $\{E, C\}$, generating $\{E, C, D\}$.

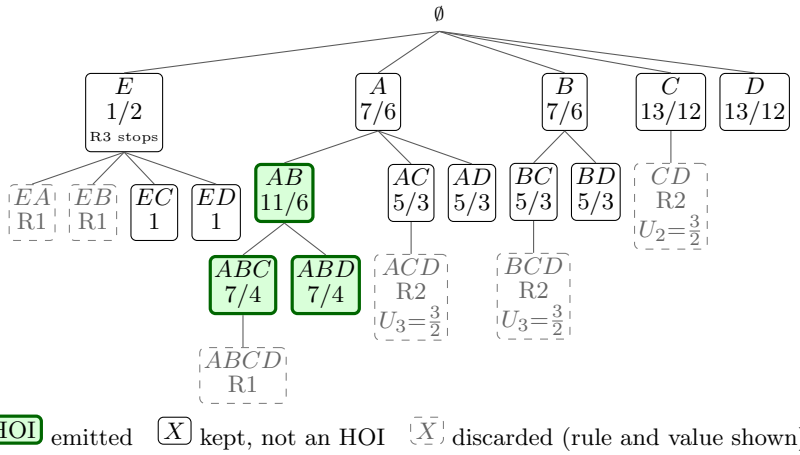


Fig. 2 Search tree of AURA-HOI on the database of Table 1 with $\alpha = 0.35$ ($\xi = 7/4$, $\sigma = 2$) and item order $E \prec A \prec B \prec C \prec D$ (Example 3). Itemsets are written without braces; the number in a kept node is its occupancy, and a discarded node shows the rule that discarded it. The discarded nodes EA, EB and $ABCD$ have support $1 < \sigma$. The children of $\{E\}$ are generated, but rule R3 does not recurse into them, since $(|\{E\}| + 2) \cdot S(\{E\}) = 3/2 < \xi$. The nodes ACD, BCD and CD pass the transaction-length bound (UBO = 2, 2 and 3) but are discarded by the length-capped bound U_K of rule R2. The search performs 15 candidate intersections and emits exactly the three HOIs, two of which have occupancy equal to ξ.

5.3 Correctness

Theorem 7 (Correctness and completeness) *Algorithm 1 emits every HOI exactly once and emits nothing else.*

Proof Soundness. An itemset is emitted only after the exact test $|Y| \cdot S(Y) \geq \xi$, that is, $\mathrm{occ}(Y) \geq \xi$.

Uniqueness. The prefix-class enumeration generates each itemset at most once [5], and pruning only removes itemsets from it.

Completeness. Let H be an HOI with items $h_1 \prec \cdots \prec h_t$ in the item order, and let $P_s = \{h_1, \ldots, h_s\}$ for $0 \leq s \leq t$. Every subset Z of H satisfies $\mathrm{supp}(Z) \geq \mathrm{supp}(H) \geq \sigma$ (Proposition 1, Lemma 2) and $U_K(Z) \geq \mathrm{occ}(H) \geq \xi$ for every $K \geq |H|$ (Theorem 3); in particular $\mathrm{UBO}(Z) \geq \xi$. We prove by induction on $s = 0, \ldots, t-1$ the following statement I_s: MINE is called on a class E with prefix P_s that contains the node $P_s \cup \{h\}$ for every $h \in H \setminus P_s$ (for $s = 0$, E is the root class E_0, and the singletons of E_0 that are HOIs are emitted when E_0 is built).

I_0 holds: for every $h \in H$, $\mathrm{supp}(\{h\}) \geq \sigma$ and $\mathrm{UBO}(\{h\}) \geq \xi$, so $\{h\} \in E_0$.

Assume I_s for some $s < t$, and let $X = P_{s+1} = X_i$ be the node of E that adds h_{s+1}. The nodes $P_s \cup \{h\}$, $h \in H \setminus P_{s+1}$, come after X_i in E, because $h \succ h_{s+1}$; hence $r_i \geq t - s - 1$ and $|H| = |X| + (t - s - 1) \leq |X| + r_i$.

- If $s + 1 = t$, then $X = H$ was emitted when its node was created, as a member of E_0 or as a child that passed the exact test.
- Otherwise, R3 does not skip X: $(|X| + r_i) \cdot S(X) \geq U_{|X|+r_i}(X) \geq \mathrm{occ}(H) \geq \xi$ by Proposition 4 and Theorem 3, and the closed form used for equal lengths is equal to

$U_{|X|+r_i}(X)$ (Proposition 5). For every $h \in H \setminus P_{s+1}$, the candidate $Y = X \cup \{h\}$ is a subset of H, so R1 keeps it, and since $|H| \leq |X|+r_i = |Y|+r_i-1$, R2 keeps it as well. Hence the set C of surviving children contains these $t-s-1$ nodes. If $t-s-1 = 1$, then $Y = H$ is one of them and is emitted when it is created. If $t-s-1 \geq 2$, then $|C| \geq 2$ and, because $T(H) \subseteq T(X)$, $(|X|+|C|) \cdot S(X) \geq |H| \cdot S(H) = \mathrm{occ}(H) \geq \xi$, so MINE is called on C, whose prefix is P_{s+1}: this is I_{s+1}.

Since s increases at each step, H is emitted after at most t steps. □

5.4 Memory management and complexity

The TID-lists of the children of X are needed only while the subtree of X is mined. AURA-HOI therefore allocates them from an arena used as a stack: it records the current top before generating the children (ARENAMARK) and restores it afterwards (ARENAREWIND). The arena is a sequence of chunks; when the current chunk is full, the next one is used, allocated on first use with at least twice the size of the previous one, so that pointers to earlier lists remain valid and chunks are reused after a rewind. This replaces one heap allocation and one release per TID-list by a pointer increment. The small array of node headers of each class is still allocated on the heap. At any time the arena holds the TID-lists of the classes on the current path of the search, that is, at most $d \cdot k_{\max}$ lists of at most n identifiers each, where d is the depth of the path and $k_{\max}$ the largest class size. This is the same asymptotic space as any depth-first TID-list miner; the arena reduces allocation cost and fragmentation, not the asymptotic space.

Processing a candidate $Y = X \cup \{x_j\}$ costs $O(|T(X)| + |T(X_j)|)$ for the merge-based intersection plus $O(|T(Y)|)$ for $S(Y)$ and $U_K(Y)$, which are computed in one pass over the result. Rule R3 costs $O(1)$ per node. The number of candidates depends on the data and on how much the rules prune, which Section 6 measures.

6 Experimental evaluation

6.1 Setup

Implementations.

HEP, DFHOI and AURA-HOI are implemented in C in the same code base, share the same data loader, and were compiled with GCC 13.3.0 at optimization level `-O2`. HEP and DFHOI are our implementations of the published algorithms. HEP [7] generates candidates level by level by joining $(k-1)$-itemsets that share their first $k-2$ items, sorts transactions by increasing length and items by increasing support [7, Section 3.2.3], and keeps a candidate when its support and UBO reach ξ. DFHOI [8] searches prefix equivalence classes depth-first and applies its two pruning conditions: a candidate whose support is below ξ is discarded before UBO is computed, and when all transactions have the same length, UBO is replaced by the support [8, Theorem 3]. The DFHOI paper does not state an item order, and its examples use alphabetical order; we use increasing support, as HEP does, and AURA-HOI uses the same order. The early termination of the UBO scan mentioned in [7, Section 3.2.3] is not implemented in any of the three programs. All three decide $\mathrm{occ}(X) \geq \xi$ exactly:

$\xi = \alpha n$ is represented as a fraction, and when the double-precision value of $\mathrm{occ}(X)$ is within $10^{-7}(1+\xi)$ of ξ, the comparison is repeated with arbitrary-precision integers over the common denominator of the lengths of the supporting transactions. Pruning tests compare bounds with $\xi - 10^{-7}(1+\xi)$, which can only keep more candidates. Our implementations of HEP and DFHOI return the numbers of HOIs reported in the original papers, with one exception that we examine in Section 6.3.

Platform and protocol.

All experiments ran on one workstation: an Intel Core i7-10700 processor (8 cores, 16 hardware threads, base frequency 2.9 GHz, maximum turbo frequency 4.8 GHz; 32 KiB L1 data cache and 256 KiB L2 cache per core, 16 MiB shared L3 cache) on an ASUS ROG STRIX B460-H GAMING motherboard, with 32 GiB of DDR4 memory and an NVMe solid-state drive, running Ubuntu 24.04.5 LTS with Linux kernel 6.8. CPU frequency was managed by the `intel_pstate` driver with the `powersave` governor and turbo enabled, the default settings of the system. Every run used one thread and was pinned to one core with `taskset`. Runtime is the duration of the mining phase, measured with a monotonic clock around the call of the algorithm; reading the input file is excluded, and the sorting of transactions and the construction of the initial TID-lists, which each algorithm performs itself, are included. Each run was limited to 600 s of wall-clock time and 20 GiB of address space. A configuration was repeated five times if its first run took less than 1 s, three times if it took less than 60 s, and once otherwise; we report the median, so runs longer than 60 s are single measurements. For each dataset the thresholds were run from the largest to the smallest, and an algorithm that exceeded a limit was not run at smaller thresholds, since every pruning rule of the three algorithms compares a bound with ξ, so a smaller threshold can only enlarge the search. The numbers of HOIs and of candidate intersections did not vary between repetitions.

Memory.

All three programs share the same executable, whose resident size after loading the data is dominated by linked libraries and is identical for every algorithm. We therefore report the memory attributable to mining: just before the mining phase, we record the resident set size and reset the kernel's peak-resident-size counter (`VmHWM`) by writing `5` to `/proc/self/clear_refs` [32]; the reported value is the peak resident size during mining minus the recorded value.

Search effort.

We count candidate intersections, that is, the number of times an algorithm intersects two TID-lists to form a candidate. This is the unit of work of all three algorithms, and it is the quantity that the bounds of Section 4 reduce.

6.2 Datasets

We use the datasets and threshold ranges of Deng [7, Section 4] (Table 2): `mushroom`, derived from the UCI Mushroom data [33]; `retail`, market baskets of a Belgian

Table 2 Datasets. Length gives the minimum and maximum transaction length (a single value means that all transactions have the same length); density is the average length divided by the number of items.

Dataset	Type	n	Items	Avg. len.	Length	Density (%)	#α	α range
`mushroom`	dense	8,416	119	23.0	23	19.33	5	0.04–0.12
`accidents`	dense	340,183	468	33.8	18–51	7.22	5	0.1–0.3
`chess`	dense	3,196	75	37.0	37	49.33	5	0.4–0.48
`pumsb`	dense	49,046	2,113	74.0	74	3.50	5	0.78–0.86
`retail`	sparse	88,162	16,470	10.3	1–76	0.06	5	0.0001–0.0005
`kosarak`	sparse	990,002	41,270	8.1	1–2497	0.02	5	0.002–0.01
`T10I4D100K`	sparse	100,000	870	10.1	1–29	1.16	5	0.0002–0.001

supermarket [34]; `accidents`, traffic accident records [35]; `kosarak`, anonymized clickstream data of a Hungarian online news portal, provided by Ferenc Bodon to the FIMI repository [36]; and `T10I4D100K`, a synthetic dataset generated with the IBM generator [2]. Deng also used `T40I10D100K`; it is not part of the dataset collection we used, and we did not include it. Because only two of these datasets are dense, we add the two dense datasets of the DFHOI evaluation that Deng did not use, `chess` and `pumsb`, both distributed by the FIMI repository [36], with the thresholds of [8, Table 8]: 48% to 40% for `chess` and 86% to 78% for `pumsb`. The files were obtained from the SPMF library [37, 38]. Its version of `mushroom` has 8,416 transactions, whereas the FIMI version used by Deng has 8,124; it is the version used in the evaluation of DFHOI [8, Table 8]. Its longest `kosarak` transaction has 2,497 distinct items, against 2,498 in [7, Table 2]. As in [7], the threshold is given as a percentage α of the number of transactions. For each dataset we use five thresholds spanning the range of [7, Table 3]: 12% to 4% for `mushroom`, 0.05% to 0.01% for `retail`, 30% to 10% for `accidents`, 1% to 0.2% for `kosarak` and 0.1% to 0.02% for `T10I4D100K`. We call `mushroom`, `accidents`, `chess` and `pumsb` dense and the other three sparse, following their density in Table 2. The distinction that matters for Corollary 6 is the number of items that reach the support threshold compared with the length of the transactions. At the middle threshold of each dataset, 60 items of `mushroom`, 48 of `accidents`, 39 of `chess` and 25 of `pumsb` reach it, against average transaction lengths of 23, 33.8, 37 and 74; on the sparse datasets, 5,382 items of `retail`, 116 of `kosarak` and 830 of `T10I4D100K` reach it, against average lengths of 10.3, 8.1 and 10.1. All transactions of `mushroom`, `chess` and `pumsb` have the same length, which is the case covered by Proposition 5. 22 of the 35 configurations have at least one HOI. The 13 exceptions are the three largest thresholds on `accidents` and all thresholds on `chess` and `pumsb`, where the DFHOI study also reports no HOI. On such configurations a miner must prove that no HOI exists, and we report them separately.

6.3 Correctness

Exact oracle.

We compared every program with an exact oracle on 900 random databases in three suites of 300: transactions of varying length (4 to 11 items, 5 to 30 transactions), transactions of a single length, and a suite designed to produce exact ties, in which transaction lengths are taken from $\{1, 2, 4, 5, 8, 10\}$ and n from $\{4, 5, 8, 10, 16, 20, 25\}$,

so that $\mathrm{occ}(X)/n$ is a terminating decimal. For each database we used the thresholds $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.4, 0.5\}$ and three thresholds placed at the occupancy of a randomly chosen itemset X: $\alpha = \mathrm{occ}(X)/n$, written exactly in the third suite and rounded to 12 significant digits in the other two, so that ξ lies within about $10^{-12}n$ of $\mathrm{occ}(X)$; values of α outside $(0, 1)$ were discarded. This gives 8,073 cases, in 2,121 of which some itemset has occupancy exactly ξ. The oracle enumerates every itemset contained in a transaction and evaluates its occupancy in rational arithmetic. In every case, the set of itemsets written by each of 7 programs (HEP, DFHOI, AURA-HOI and its four ablation variants of Section 6.6) is equal to the set returned by the oracle.

Benchmarks.

For each of the 34 configurations of Table A1 completed by at least two algorithms, and for the 7 configurations of the ablation study, every completed program was run once more and wrote its HOIs. The sorted outputs have identical SHA-256 digests in every configuration.

Published counts.

The numbers of HOIs returned by the three programs are equal to those reported by Deng [7, Table 3] for `retail` (0.05%, 0.03%, 0.01%), `kosarak` (1%, 0.6%, 0.2%) and `T10I4D100K` (0.1%, 0.06%, 0.02%), and for `accidents` at 30% (0) and 10% (5,535). They are also equal to those reported for DFHOI [8, Tables 7 and 8] for `T10I4D100K` at all five thresholds of Table A1, for `retail` at 0.04%, for `chess` and `pumsb` at all five thresholds (no HOI), and for `mushroom` at the thresholds used there (13%, 11%, 9%, 7%, 5%: 5, 198, 3,266, 27,123 and 93,266 HOIs), which we ran separately. Deng's counts for `mushroom` refer to the FIMI version of the dataset and are not comparable. One count differs: for `accidents` at 20%, Deng reports 15 HOIs, whereas the three programs return none (and two at 15%). The numbers of frequent itemsets that Deng reports for `accidents` at 30% and 20% (149,545 and 889,883) are equal to those of our file, so the data are the same. A separate program that shares no code with the three implementations (`brute_check.py`, distributed with our code) enumerates all 889,883 frequent itemsets at 20% and evaluates their occupancy with integer arithmetic; it finds no HOI, and the largest occupancy is $0.1522\,n$. Run at 15%, the same program enumerates 2,676,381 frequent itemsets and finds exactly the two HOIs returned by the three implementations. We could not determine the origin of the published value.

6.4 Runtime and search effort

Table 3 summarizes the comparison per dataset, Figures 3 and 4 show runtime and search effort for every threshold, and Table A1 in the Appendix gives all values.

Dense datasets.

On `mushroom`, whose transactions all have length 23, the transaction-length bound equals the support (Proposition 5), and HEP and DFHOI perform exactly the same number of candidate intersections at every threshold (Table A1). AURA-HOI performs 85% to 98% fewer and is 4.9 to 26 times faster than DFHOI. On `accidents`,

Table 3 Summary per dataset. Finished: configurations completed within the limits. Faster: configurations in which AURA-HOI is faster than DFHOI, among those completed by both. Speedup: runtime of DFHOI divided by that of AURA-HOI (geometric mean and maximum) over the same configurations. Reduction: median reduction of candidate intersections relative to DFHOI.

		Finished				Speedup vs. DFHOI		Reduction
Dataset	Configs	HEP	DFHOI	AURA-HOI	Faster	Geo. mean	Max	(%)
`mushroom`	5	5	5	5	5/5	12.0	26.0	93.4
`accidents`	5	1	4	5	4/4	69.0	85.7	99.5
`chess`	5	5	5	5	5/5	1933	2989	100.0
`pumsb`	5	5	5	5	5/5	589	2291	100.0
`retail`	5	5	5	5	5/5	1.17	1.21	0.1
`kosarak`	5	5	5	5	4/5	1.08	1.14	7.6
`T10I4D100K`	5	5	5	5	5/5	1.17	1.21	1.2

transaction lengths vary from 18 to 51, but the transaction-length bound still prunes little: DFHOI performs between 68,644 and 1,559,815 intersections at the thresholds it completes, and AURA-HOI between 387 and 8,244, a reduction of more than 99%. At 30%, 25% and 20% no itemset has high occupancy, yet DFHOI needs 28 s to 123 s to establish this, against 0.5 s to 1.5 s for AURA-HOI. At 15%, DFHOI needs 298 s and AURA-HOI 4.6 s to find the two HOIs; at 10%, DFHOI exceeds the 600 s limit and AURA-HOI finds the 5,535 HOIs in 28 s. HEP exceeds the memory limit at 25% (Section 6.5).

On `chess` and `pumsb`, whose transactions also have a single length (37 and 74), no HOI exists at any of the thresholds of the DFHOI study. HEP and DFHOI again perform the same number of intersections, from 1,767,986 to 6,512,322 on `chess` and from 16,368 to 299,566 on `pumsb`, and DFHOI needs 2.9 s to 8.8 s and 1.1 s to 18.4 s to complete. AURA-HOI performs between 366 and 644 intersections on `chess`, and none on `pumsb`: there, 22 to 26 items reach the support threshold, so no itemset of the search has more than 26 items, and the class bound of the root class, evaluated with the closed form of Proposition 5, is at most $26/74 \cdot \mathrm{supp}(\{x\}) \leq 0.36\,n < \xi$ for every item x. Rule R3 therefore discards every node of the root class, and AURA-HOI completes in less than 0.01 s. On both datasets AURA-HOI runs in a few milliseconds, so the speedups of Table 3 (geometric means of 1,933 and 589 over DFHOI) mainly measure the time the baselines spend in their search, and we do not use them as the measure of the benefit of the bound.

Over the 6 dense configurations completed by both programs that have at least one HOI, AURA-HOI is 4.86 to 64.2 times faster than DFHOI (geometric mean 15.9); over the 13 dense configurations without HOI, it is at least 50.6 times faster.

Sparse datasets.

On `retail`, `kosarak` and `T10I4D100K`, AURA-HOI performs 0.1% to 8.3% fewer intersections than DFHOI. This is the behaviour predicted by Corollary 6: when the size limit K of rule R2 is at least the length of every supporting transaction, U_K equals the transaction-length bound. On `retail` and `T10I4D100K`, 5,382 and 830 items reach the support threshold at the middle thresholds, against longest transactions of 76 and 29 items, so the classes near the root are much longer than any transaction and this

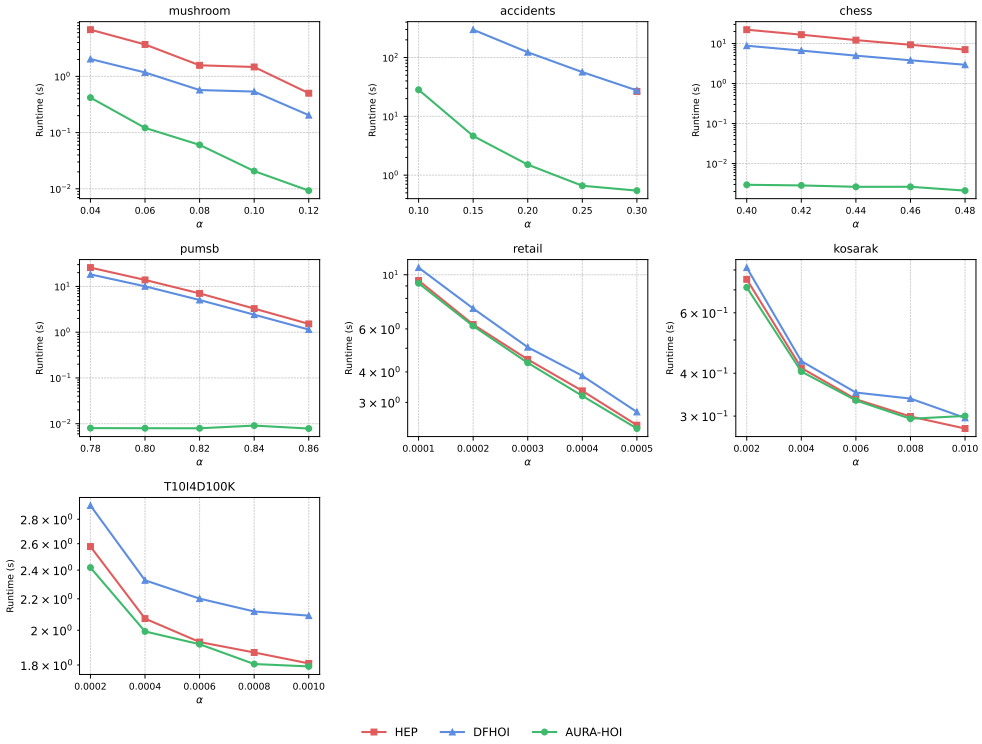


Fig. 3 Runtime (seconds, logarithmic scale) as a function of the threshold α. Missing points: the run exceeded the memory or the time limit, or was not run because the same algorithm exceeded a limit at a larger threshold.

is the usual case; rule R3 is weak for the same reason. On `kosarak` the search is small for all three programs (at most 5,769 intersections). The runtimes of the three programs are close. Compared with the faster of HEP and DFHOI in each configuration, AURA-HOI is 0.92 to 1.07 times as fast, and slower in 1 of 15 configurations (`kosarak` at 1%, 0.30 s against 0.28 s). Separately, AURA-HOI is 0.99 to 1.21 times as fast as DFHOI (geometric mean 1.14) and 0.92 to 1.07 times as fast as HEP (geometric mean 1.02). On `retail` and `T10I4D100K`, AURA-HOI takes about 15% less time than DFHOI although both perform almost the same intersections; disabling the arena does not remove this difference (Table 4), so it comes from implementation details that we did not isolate, not from the bounds.

Overall.

AURA-HOI is faster than DFHOI in 33 of the 34 configurations completed by both and faster than HEP in 30 of 31. A two-sided Wilcoxon signed-rank test [39, 40] on the logarithms of the paired runtimes gives $W = 1$, $p = 2.3 \times 10^{-10}$ ($N = 34$) against DFHOI and $W = 15$, $p = 1.3 \times 10^{-7}$ ($N = 31$) against HEP. These tests treat the

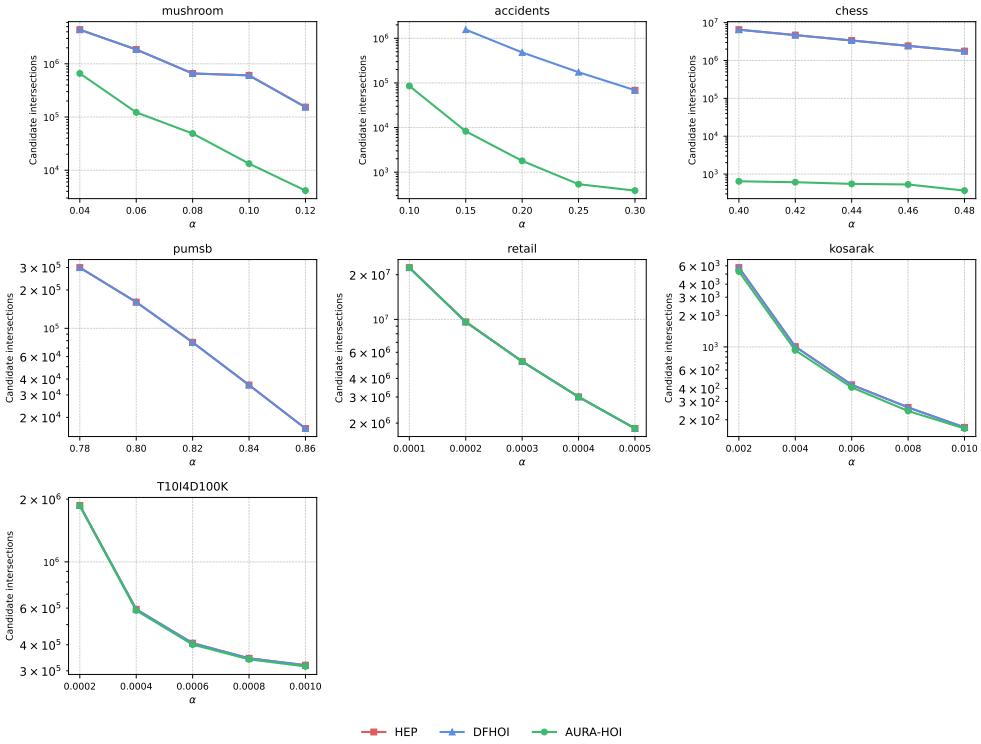


Fig. 4 Candidate intersections (logarithmic scale) as a function of the threshold α.

configurations as independent, although the thresholds of one dataset are not; the per-dataset values of Table 3 show that the differences are large on the dense datasets and small on the sparse ones. Over the 98 measurements that were repeated, the median relative spread (largest minus smallest repetition, divided by the median) is 2.0%. The largest spread, 61%, comes from a single slow repetition of a run of 0.12 s; this is why we report medians.

6.5 Memory

Figure 5 and Table A1 report the peak memory attributable to mining. HEP stores the occupancy lists of a whole level of its breadth-first search; it uses up to 3.6 GB on `mushroom`, 14.1 GB on `chess`, 17.8 GB on `accidents` at 30% and 19,285 MB on `pumsb` at 78%, close to the 20 GiB limit, which it exceeds on `accidents` at 25%. The two depth-first programs keep only the TID-lists of the current path. AURA-HOI uses at most 89.1 MB (median 5.0 MB); its memory is between 0.22 and 1.11 times that of DFHOI, and lower in 25 of the 34 configurations. The smallest ratios occur where AURA-HOI prunes almost the whole search: 0.22 on `pumsb` and 0.56 on `accidents` at 30%. As stated in Section 5, the arena does not change the asymptotic space of the depth-first search.

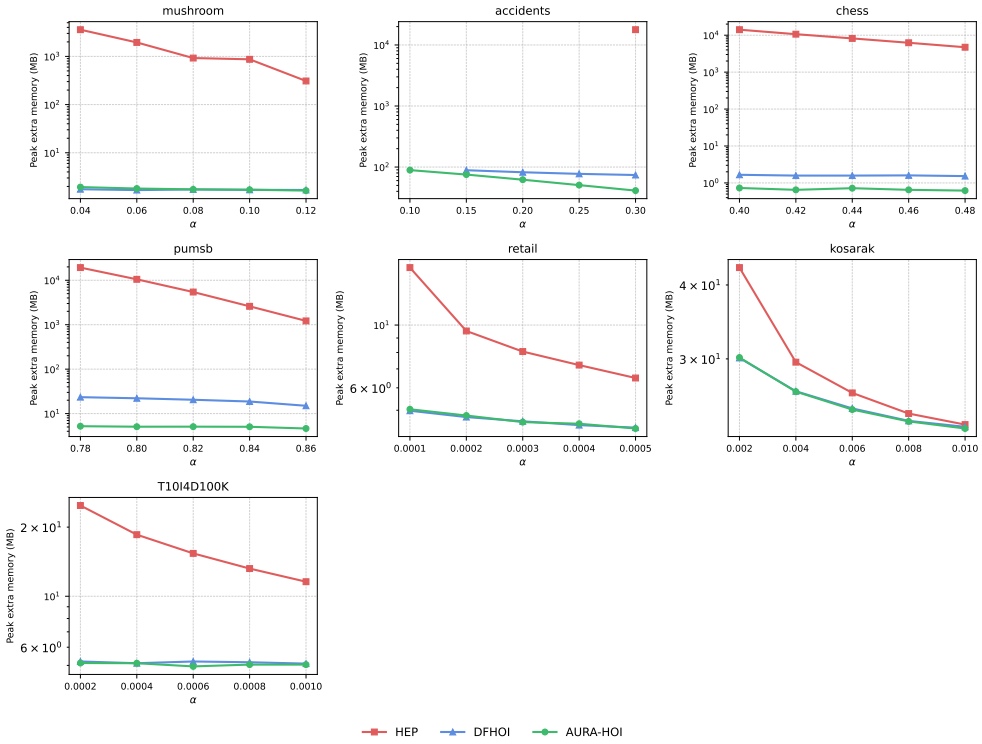


Fig. 5 Peak memory attributable to mining (MB, logarithmic scale).

Table 4 Ablation study: runtime in seconds and, in parentheses, candidate intersections. ML: memory limit exceeded.

Dataset	α	Full	No tail cap	No class bound	No residual	No arena
`mushroom`	0.08	0.059 (48,890)	0.062 (50,905)	0.082 (75,947)	0.425 (657,947)	0.076 (48,890)
`accidents`	0.2	1.47 (1,794)	1.50 (1,839)	3.47 (6,539)	91.7 (482,214)	1.54 (1,794)
`chess`	0.44	0.003 (551)	0.003 (551)	0.007 (2,985)	3.57 (3,375,255)	0.003 (551)
`pumsb`	0.82	0.008 (0)	0.008 (0)	0.025 (300)	2.43 (77,654)	0.008 (0)
`retail`	0.0003	4.43 (5,204,094)	4.42 (5,204,132)	4.36 (5,208,507)	4.42 (5,209,673)	4.42 (5,204,094)
`kosarak`	0.006	0.317 (409)	0.317 (417)	0.317 (417)	0.324 (433)	0.320 (409)
`T10I4D100K`	0.0006	1.86 (401,045)	1.86 (401,153)	1.85 (405,236)	1.86 (407,345)	1.85 (401,045)

6.6 Ablation study

To measure the contribution of each component, we disabled them one at a time: *no tail cap* uses the transaction-length bound (6) instead of U_K in rule R2; *no class bound* removes rule R3; *no residual* does both, so that only transaction lengths are used, as in DFHOI; and *no arena* allocates every TID-list on the heap. Table 4 reports runtime and candidate intersections.

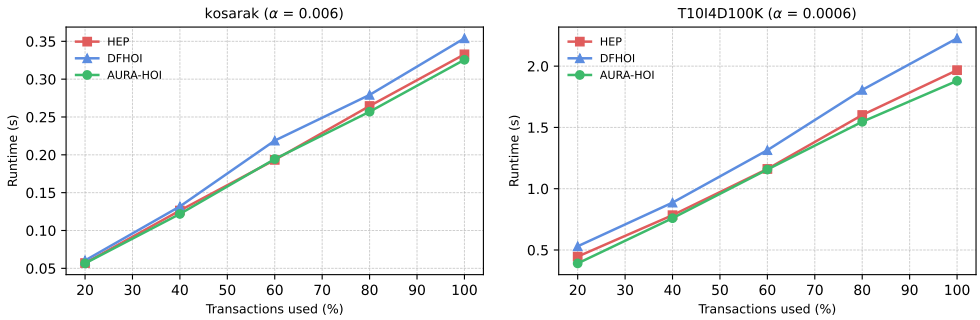


Fig. 6 Runtime on growing prefixes of the database at a fixed threshold.

On the dense datasets the components are complementary. Removing both residual rules (*no residual*) leaves only the transaction-length bound, and AURA-HOI then performs exactly as many intersections as DFHOI (657,947 on `mushroom`, 482,214 on `accidents`, 3,375,255 on `chess` and 77,654 on `pumsb`, Table A1) and becomes 7.2, 62, about 1,400 and about 300 times slower. Removing only one of the two rules costs much less, because each rule prunes most of what the other prunes: without the tail cap, intersections increase by 4.1% on `mushroom` and 2.5% on `accidents`; without the class bound they increase by 55% and by a factor of 3.6, and the runtime by factors of 1.4 and 2.4. On `chess` and `pumsb`, removing the class bound raises the intersections from 551 to 2,985 and from 0 to 300, whereas removing the tail cap changes nothing: when all transactions have the same length, rule R3 is evaluated with the closed form of Proposition 5, which already caps the residual length at L, so on these datasets the class bound subsumes the node bound. The arena reduces the runtime by 22% on `mushroom`, where TID-lists are short and allocations are frequent relative to the work per candidate, and by 4% on `accidents`; on the sparse datasets, no variant changes the runtime by more than 3%, consistently with the small reductions of Section 6.4.

6.7 Scalability

To measure how runtime grows with the number of transactions, we ran the three programs on the first 20%, 40%, 60%, 80% and 100% of the transactions of `kosarak` and `T10I4D100K`, with the middle threshold of Table A1 expressed as the same percentage of the prefix length (Figure 6). The runtimes of the three programs grow almost linearly: from 20% to 100% of the data, the runtime of AURA-HOI grows by a factor of 5.8 on `kosarak` and 4.8 on `T10I4D100K`, and the relative order of the three programs does not change. These two datasets are sparse, so the scalability study reflects the cost of building and intersecting TID-lists rather than the effect of the bounds.

6.8 Limitations

- The bound helps only when the residual length of a branch is smaller than the lengths of the supporting transactions (Corollary 6), which is the case on the dense datasets. On the sparse datasets, AURA-HOI is not substantially faster than HEP.

On the dense datasets, 13 of the 19 configurations completed by both AURA-HOI and DFHOI have no HOI; the thresholds are those of the original studies, and the gains on configurations with HOIs rest on `mushroom` and `accidents`.

- The comparison uses our C implementations of HEP and DFHOI, not the authors' code. Their numbers of HOIs agree with the published ones, except for the value discussed in Section 6.3, but their runtimes depend on implementation choices, as the difference between DFHOI and AURA-HOI on `retail` shows. The numbers of candidate intersections do not depend on the implementation or on the machine.
- We used five of the six datasets of Deng and two datasets of the DFHOI evaluation, the SPMF version of `mushroom`, and one machine. All runs used a single thread.
- The arena reduces allocation cost but not the asymptotic space of the search, and AURA-HOI reports every HOI; on databases with very many HOIs, the size of the output dominates.

7 Conclusion

The transaction-length bound used by the existing exact miners of high-occupancy itemsets is weak on dense databases, where it is close to, or equal to, the support. We presented a length-capped residual bound that also uses the largest itemset that a branch of a depth-first search can still generate, proved that it is safe and never weaker than the transaction-length bound, and characterized exactly when it coincides with it: when the branch can still produce itemsets as long as the supporting transactions. Built on this bound, AURA-HOI is an exact miner, and we showed that a natural shortcut in its residual length loses results. All programs agree with an exact oracle on 8,073 randomized cases, including 2,121 with exact ties, and with each other on every benchmark configuration. On the four dense datasets of the evaluations of HEP and DFHOI, AURA-HOI performs at least 84.9% fewer candidate intersections than DFHOI. Where HOIs exist it is 4.86 to 64.2 times faster, and it completes the smallest threshold on `accidents`, which DFHOI does not complete within 600 s and HEP cannot run within 20 GiB; where none exists, it establishes this in at most 1.51 s. On the sparse datasets, as the characterization predicts, the bound prunes little, and AURA-HOI runs 0.92 to 1.07 times as fast as the faster of the two baselines.

AURA-HOI is therefore suited to dense data, and on sparse data its runtime stays close to that of the baselines. Two directions follow. First, the thresholds of the earlier studies leave `chess` and `pumsb` without HOIs; an evaluation at lower thresholds on these and further dense databases would measure the benefit of the bound where many HOIs exist. Second, sparse data call for a bound that does not rely on the size of the branch; a per-transaction residual bound, in the spirit of the remaining-utility bound of HUOPM [13], is a candidate, and Section 2 shows that neither of the two bounds dominates the other, so their minimum can be used.

Acknowledgements. Not applicable.

Declarations

- **Funding:** Funding not applicable. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.
- **Competing interests:** The author declares no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** The benchmark datasets are publicly available from the SPMF library [37, 38]; `retail`, `accidents`, `kosarak` and `T10I4D100K` are also distributed by the FIMI repository [36]. The raw measurements (one file per experiment) are available from the corresponding author on request.
- **Materials availability:** Not applicable.
- **Code availability:** The source code of the three implementations and the scripts that reproduce every experiment are available from the corresponding author on reasonable request.
- **Author contribution:** Q. Van conceived the method, proved the results, implemented the algorithms, performed the experiments and wrote the manuscript.

References

[1] Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, pp. 207–216. ACM, New York, NY, USA (1993). https://doi.org/10.1145/170035.170072

[2] Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), pp. 487–499 (1994). https://www.vldb.org/conf/1994/P487.PDF

[3] Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp. 1–12. ACM, New York, NY, USA (2000). https://doi.org/10.1145/342009.335372

[4] Zaki, M.J., Parthasarathy, S., Ogihara, M., Li, W.: New algorithms for fast discovery of association rules. In: Proceedings of the Third International Conference on Knowledge Discovery and Data Mining (KDD), pp. 283–286. AAAI Press, Menlo Park, CA, USA (1997). https://cdn.aaai.org/KDD/1997/KDD97-060.pdf

[5] Zaki, M.J.: Scalable algorithms for association mining. IEEE Transactions on Knowledge and Data Engineering **12**(3), 372–390 (2000) https://doi.org/10.1109/69.846291

[6] Tang, L., Zhang, L., Luo, P., Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management

(CIKM), pp. 75–84. ACM, New York, NY, USA (2012). https://doi.org/10.1145/2396761.2396775

[7] Deng, Z.-H.: Mining high occupancy itemsets. Future Generation Computer Systems **102**, 222–229 (2020) https://doi.org/10.1016/j.future.2019.07.039

[8] Nguyen, L.T.T., Mai, T., Pham, G.-H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowledge-Based Systems **267**, 110441 (2023) https://doi.org/10.1016/j.knosys.2023.110441

[9] Datta, S., Mali, K., Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arabian Journal for Science and Engineering **47**(2), 2061–2075 (2022) https://doi.org/10.1007/s13369-021-06075-8

[10] Kim, H., Ryu, T., Lee, C., Kim, H., Truong, T., Fournier-Viger, P., Pedrycz, W., Yun, U.: Mining high occupancy patterns to analyze incremental data in intelligent systems. ISA Transactions **131**, 460–475 (2022) https://doi.org/10.1016/j.isatra.2022.05.003

[11] Tran, T.-N., Truong Hoang, V., Truong, T.-C., Voznak, M.: A hierarchical set-enumeration tree enabling high occupancy item set mining and the use of an adaptive occupancy threshold. Applied Intelligence **55**(3), 205 (2025) https://doi.org/10.1007/s10489-024-06166-7

[12] Yıldırım, İ.: Mining top-k high occupancy itemsets. Black Sea Journal of Engineering and Science **8**(6), 1723–1730 (2025) https://doi.org/10.34248/bsengineering.1744061

[13] Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Yu, P.S.: HUOPM: High-utility occupancy pattern mining. IEEE Transactions on Cybernetics **50**(3), 1195–1208 (2020) https://doi.org/10.1109/TCYB.2019.2896267

[14] Kumar, M.J.K., Rana, D.: HAUOPM: High average utility occupancy pattern mining. Arabian Journal for Science and Engineering **49**(3), 3397–3416 (2024) https://doi.org/10.1007/s13369-023-07971-x

[15] Cho, M., Kim, H., Park, S., Kim, D., Kim, D., Yun, U.: Advanced approach for mining utility occupancy patterns in incremental environment. Knowledge-Based Systems **306**, 112713 (2024) https://doi.org/10.1016/j.knosys.2024.112713

[16] Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM), pp. 55–64. ACM, New York, NY, USA (2012). https://doi.org/10.1145/2396761.2396773

[17] Zida, S., Fournier-Viger, P., Lin, J.C.-W., Wu, C.-W., Tseng, V.S.: EFIM: A

fast and memory efficient algorithm for high-utility itemset mining. Knowledge and Information Systems **51**(2), 595–625 (2017) https://doi.org/10.1007/s10115-016-0986-0

[18] Zaki, M.J., Gouda, K.: Fast vertical mining using diffsets. In: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 326–335. ACM, New York, NY, USA (2003). https://doi.org/10.1145/956750.956788

[19] Deng, Z., Wang, Z., Jiang, J.: A new algorithm for fast mining frequent itemsets using N-lists. Science China Information Sciences **55**(9), 2008–2030 (2012) https://doi.org/10.1007/s11432-012-4638-z

[20] Deng, Z.-H., Lv, S.-L.: PrePost+: An efficient N-lists-based algorithm for mining frequent itemsets via Children–Parent equivalence pruning. Expert Systems with Applications **42**(13), 5424–5432 (2015) https://doi.org/10.1016/j.eswa.2015.03.004

[21] Pasquier, N., Bastide, Y., Taouil, R., Lakhal, L.: Discovering frequent closed itemsets for association rules. In: Database Theory – ICDT'99. Lecture Notes in Computer Science, vol. 1540, pp. 398–416. Springer, Berlin, Heidelberg (1999). https://doi.org/10.1007/3-540-49257-7_25

[22] Zaki, M.J., Hsiao, C.-J.: CHARM: An efficient algorithm for closed itemset mining. In: Proceedings of the 2002 SIAM International Conference on Data Mining, pp. 457–473. SIAM, Philadelphia, PA, USA (2002). https://doi.org/10.1137/1.9781611972726.27

[23] Uno, T., Kiyomi, M., Arimura, H.: LCM ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets. In: Proceedings of the IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI'04). CEUR Workshop Proceedings, vol. 126 (2004). https://ceur-ws.org/Vol-126/uno.pdf

[24] Bayardo, R.J.: Efficiently mining long patterns from databases. ACM SIGMOD Record **27**(2), 85–93 (1998) https://doi.org/10.1145/276305.276313

[25] Aggarwal, C.C., Han, J. (eds.): Frequent Pattern Mining. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07821-2

[26] Fournier-Viger, P., Lin, J.C.-W., Vo, B., Chi, T.T., Zhang, J., Le, H.B.: A survey of itemset mining. WIREs Data Mining and Knowledge Discovery **7**(4), 1207 (2017) https://doi.org/10.1002/widm.1207

[27] Luna, J.M., Fournier-Viger, P., Ventura, S.: Frequent itemset mining: A 25 years review. WIREs Data Mining and Knowledge Discovery **9**(6), 1329 (2019) https://doi.org/10.1002/widm.1329

[28] Seno, M., Karypis, G.: Finding frequent patterns using length-decreasing support constraints. Data Mining and Knowledge Discovery **10**(3), 197–228 (2005) https://doi.org/10.1007/s10618-005-0364-0

[29] Fournier-Viger, P., Lin, J.C.-W., Truong-Chi, T., Nkambou, R.: A survey of high utility itemset mining. In: High-Utility Pattern Mining, pp. 1–45. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-04921-8_1

[30] Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Tseng, V.S., Yu, P.S.: A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering **33**(4), 1306–1327 (2021) https://doi.org/10.1109/TKDE.2019.2942594

[31] Fournier-Viger, P., Wu, C.-W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Foundations of Intelligent Systems (ISMIS 2014). Lecture Notes in Computer Science, vol. 8502, pp. 83–92. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-08326-1_9

[32] The Linux Kernel Documentation: The /proc Filesystem. https://www.kernel.org/doc/html/latest/filesystems/proc.html. Accessed 14 October 2026 (2026)

[33] Mushroom. UCI Machine Learning Repository (1981). https://doi.org/10.24432/C5959T

[34] Brijs, T., Swinnen, G., Vanhoof, K., Wets, G.: Using association rules for product assortment decisions. In: Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 254–260. ACM, New York, NY, USA (1999). https://doi.org/10.1145/312129.312241

[35] Geurts, K., Wets, G., Brijs, T., Vanhoof, K.: Profiling of high-frequency accident locations by use of association rules. Transportation Research Record **1840**(1), 123–130 (2003) https://doi.org/10.3141/1840-14

[36] Goethals, B., Zaki, M.J.: Frequent Itemset Mining Dataset Repository. http://fimi.uantwerpen.be/. Accessed 14 October 2026 (2004)

[37] Fournier-Viger, P.: SPMF: An Open-Source Data Mining Library – Datasets. https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php. Accessed 14 October 2026 (2026)

[38] Fournier-Viger, P., Lin, J.C.-W., Gomariz, A., Gueniche, T., Soltani, A., Deng, Z., Lam, H.T.: The SPMF open-source data mining library version 2. In: Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2016). Lecture Notes in Computer Science, vol. 9853, pp. 36–40. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46131-1_8

[39] Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin

1(6), 80–83 (1945) https://doi.org/10.2307/3001968

[40] Demšar, J.: Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research **7**, 1–30 (2006)

Appendix A Results for every configuration

Table A1: Runtime (s; median of the repetitions; fastest in bold), candidate intersections and peak memory attributable to mining (MB) for every configuration. ML: memory limit exceeded; TL: time limit exceeded; ML*: not run because the same algorithm exceeded the memory limit at a larger threshold.

			Runtime (s)			Intersections		Memory (MB)		
Dataset	α	HOIs	HEP	DFHOI	AURA-HOI	DFHOI	AURA-HOI	HEP	DFHOI	AURA-HOI
mushroom	0.12	31	0.502	0.204	**0.009**	153,538	4,137	309	1.7	1.6
	0.1	955	1.47	0.536	**0.021**	606,050	13,294	870	1.7	1.7
	0.08	15,292	1.57	0.571	**0.060**	657,947	48,890	924	1.7	1.7
	0.06	48,316	3.70	1.17	**0.121**	1,857,321	122,239	1949	1.7	1.8
	0.04	335,388	6.76	2.03	**0.419**	4,378,698	659,396	3583	1.8	1.9
accidents	0.3	0	26.5	27.6	**0.546**	68,644	387	17793	74.0	41.1
	0.25	0	ML	56.6	**0.660**	174,012	538	–	77.3	50.6
	0.2	0	ML*	123	**1.51**	482,214	1,794	–	82.2	62.1
	0.15	2	ML*	298	**4.64**	1,559,815	8,244	–	88.7	75.3
	0.1	5,535	ML*	TL	**28.3**	–	85,555	–	–	89.1
chess	0.48	0	7.03	2.94	**0.002**	1,767,986	366	4724	1.5	0.6
	0.46	0	9.30	3.81	**0.003**	2,437,410	531	6242	1.6	0.7
	0.44	0	12.2	4.97	**0.003**	3,375,255	551	8188	1.6	0.7
	0.42	0	16.6	6.66	**0.003**	4,682,691	609	10659	1.6	0.7
	0.4	0	22.1	8.80	**0.003**	6,512,322	644	14106	1.7	0.7
pumsb	0.86	0	1.52	1.14	**0.008**	16,368	0	1219	15.0	4.6
	0.84	0	3.28	2.41	**0.009**	35,904	0	2594	18.7	5.0
	0.82	0	7.04	5.05	**0.008**	77,654	0	5440	20.5	5.1
	0.8	0	13.9	10.1	**0.008**	160,633	0	10507	22.1	5.1
	0.78	0	25.8	18.4	**0.008**	299,566	0	19285	23.4	5.2
retail	0.0005	1,170	2.42	2.74	**2.34**	1,841,381	1,839,104	6	4.3	4.3
	0.0004	1,662	3.35	3.86	**3.19**	3,005,287	3,001,972	7	4.4	4.5
	0.0003	2,620	4.50	5.05	**4.36**	5,209,673	5,204,094	8	4.6	4.5
	0.0002	5,108	6.26	7.28	**6.17**	9,616,670	9,605,279	10	4.7	4.8
	0.0001	14,202	9.47	10.7	**9.23**	22,342,757	22,284,441	16	5.0	5.0
kosarak	0.01	31	**0.276**	0.296	0.300	169	165	23	23.0	22.9
	0.008	48	0.299	0.337	**0.295**	263	243	24	23.6	23.5
	0.006	71	0.336	0.351	**0.333**	433	409	26	24.7	24.6
	0.004	117	0.414	0.434	**0.405**	1,007	928	30	26.4	26.4
	0.002	265	0.751	0.813	**0.712**	5,769	5,291	43	30.1	30.2
T10I4D100K	0.001	4,363	1.81	2.09	**1.79**	319,316	315,076	12	5.1	5.0
	0.0008	7,872	1.87	2.12	**1.81**	344,553	340,656	13	5.2	5.0
	0.0006	12,640	1.93	2.20	**1.92**	407,345	401,045	15	5.2	5.0
	0.0004	20,895	2.07	2.33	**1.99**	592,028	584,678	19	5.1	5.1
	0.0002	38,588	2.58	2.92	**2.42**	1,864,361	1,856,485	25	5.2	5.1